

Integrating REST APIs as LLM Agents with LangChain

This tutorial builds a LangChain agent that can interact with a REST API and decide **when and how to call the API based on the user's request**.

We will use **Fake Store API** (fakestoreapi.com) as the external REST service. It provides products, carts, users, and authentication endpoints, making it useful for demonstrating tool-based API integration. ([GitHub](#))

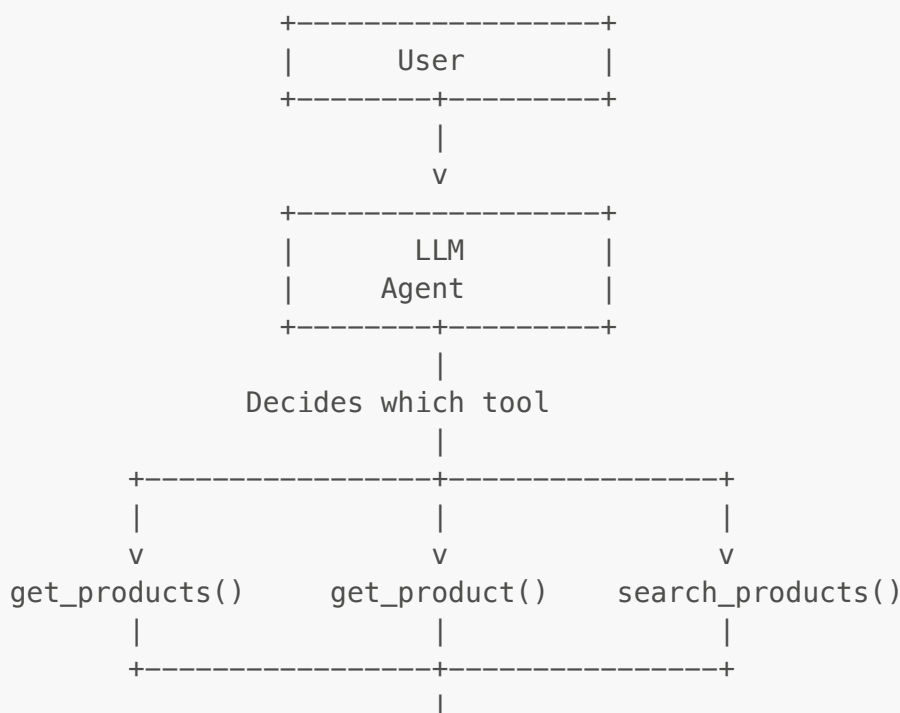
The examples use the current LangChain agent approach with `create_agent()`. LangChain's current agent API executes a loop in which the model can call tools, receive `ToolMessage` results, and continue reasoning until it produces a final response. ([LangChain Reference](#))

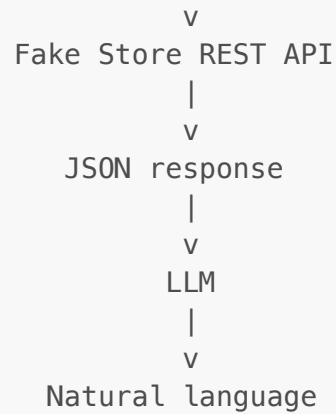
1. What we are building

We will gradually move from:



to:





The important idea is:

The REST API is not itself the agent. The REST API is exposed to the agent through LangChain tools.

LangChain tools provide the interface through which an agent interacts with external systems. The `@tool` decorator can turn a Python function into a tool, automatically deriving its input schema from the function signature. ([LangChain Reference](#))

Part 1 — Calling Fake Store API directly

Before introducing an LLM, let's understand the REST API.

Fake Store API exposes resources such as:

```
GET /products
GET /products/{id}

GET /carts
GET /carts/{id}

GET /users
GET /users/{id}
```

For example:

```
https://fakestoreapi.com/products
```

returns product information.

A typical product looks like:

```
{
  "id": 1,
  "title": "Fjallraven Backpack",
  "price": 109.95,
  "category": "men's clothing",
  "description": "...",
  "image": "..."
}
```

Part 2 — Calling the REST API using Python

Install `requests`:

```
pip install requests
```

Then:

```
import requests

url = "https://fakestoreapi.com/products"

response = requests.get(url)

print(response.status_code)

products = response.json()

for product in products:
    print(product["id"], product["title"], product["price"])
```

The flow is simply:

```
Python
  |
  | HTTP GET
  v
Fake Store API
  |
  | JSON
  v
Python
```

There is no LLM involved yet.

Part 3 — Convert the REST call into a LangChain tool

Now we turn the API operation into a tool.

```
import requests

from langchain_core.tools import tool

@tool
def get_products() -> str:
    """Get all products from the Fake Store API."""

    response = requests.get(
        "https://fakestoreapi.com/products"
    )

    response.raise_for_status()

    return response.text
```

The important part is:

```
@tool
def get_products():
```

The function is now exposed as a LangChain tool.

The docstring is also important:

```
"""Get all products from the Fake Store API."""
```

The LLM uses the tool's name, description, and argument schema to determine whether the tool is appropriate for a request. LangChain specifically recommends meaningful tool names, descriptions, and schemas because they help models select tools correctly. ([LangChain Reference](#))

Conceptually:

```
get_products
|
+-- name
|
```

```
+-- description
|
+-- input schema
|
+-- Python implementation
    |
    v
    REST API
```

Part 4 — Give the tool to an LLM

Now let's create an agent.

For OpenAI:

```
pip install -U langchain langchain-openai requests python-dotenv
```

Set:

```
OPENAI_API_KEY=your-api-key
```

Then:

```
from langchain.agents import create_agent
from langchain_openai import ChatOpenAI
from langchain_core.tools import tool

import requests

@tool
def get_products() -> str:
    """Get all products from the Fake Store API."""

    response = requests.get(
        "https://fakestoreapi.com/products"
    )

    response.raise_for_status()

    return response.text

llm = ChatOpenAI(
    model="gpt-5.6"
```

```
)

agent = create_agent(
    model=llm,
    tools=[get_products],
    system_prompt="You are a shopping assistant."
)

result = agent.invoke({
    "messages": [
        {
            "role": "user",
            "content": "Show me the available products."
        }
    ]
})

print(result["messages"][-1].content)
```

The current LangChain `create_agent()` API accepts a model and a list of tools and handles the tool-calling loop. ([LangChain Reference](#))

Part 5 — What actually happens?

Suppose the user asks:

```
Show me the available products.
```

The LLM receives something conceptually like:

```
Available tool:

get_products()
Description:
Get all products from the Fake Store API.
```

The LLM decides:

```
I need product information.

Call:
get_products()
```

LangChain executes:

```
get_products()
```

which performs:

```
requests.get(  
    "https://fakestoreapi.com/products"  
)
```

The API returns JSON.

LangChain sends that tool result back to the LLM.

Then the LLM generates the final response.

So the complete sequence is:

```
User  
|  
| "Show me the available products"  
v  
LLM  
|  
| tool_call:  
| get_products()  
v  
LangChain  
|  
| execute Python function  
v  
get_products()  
|  
| HTTP GET  
v  
Fake Store API  
|  
| JSON  
v  
get_products()  
|  
| tool result  
v  
LLM  
|  
v  
Final answer
```

This distinction is important:

The LLM does not directly execute the HTTP request.

The LLM requests that a tool be executed.

The Python function performs the HTTP request.

Part 6 — Add a parameter to the REST tool

The previous tool can only retrieve all products.

Let's expose:

```
GET /products/{id}
```

as a tool.

```
@tool
def get_product(product_id: int) -> str:
    """Get a specific product from the Fake Store API using its product
    ID."""

    url = f"https://fakestoreapi.com/products/{product_id}"

    response = requests.get(url)

    response.raise_for_status()

    return response.text
```

Notice:

```
product_id: int
```

This is important.

LangChain can derive an input schema from the function signature. ([LangChain Reference](#))

The model therefore sees something conceptually similar to:

```
{
  "name": "get_product",
  "description": "Get a specific product..."
```

```
"parameters": {  
    "product_id": {  
        "type": "integer"  
    }  
}
```

Now the user can say:

Give me details about product 5.

The LLM can generate a tool call equivalent to:

```
get_product(product_id=5)
```

LangChain executes:

```
GET https://fakestoreapi.com/products/5
```

Part 7 — Create multiple REST API tools

Now let's expose several API operations.

```
import requests  
  
from langchain_core.tools import tool  
  
BASE_URL = "https://fakestoreapi.com"  
  
@tool  
def get_products() -> str:  
    """Get all products from the store."""  
  
    response = requests.get(  
        f"{BASE_URL}/products"  
    )  
  
    response.raise_for_status()  
  
    return response.text
```

```
@tool
def get_product(product_id: int) -> str:
    """Get a product by its ID."""

    response = requests.get(
        f"{BASE_URL}/products/{product_id}"
    )

    response.raise_for_status()

    return response.text


@tool
def get_carts() -> str:
    """Get all shopping carts."""

    response = requests.get(
        f"{BASE_URL}/carts"
    )

    response.raise_for_status()

    return response.text


@tool
def get_users() -> str:
    """Get all users."""

    response = requests.get(
        f"{BASE_URL}/users"
    )

    response.raise_for_status()

    return response.text
```

Now create the agent:

```
from langchain.agents import create_agent
from langchain_openai import ChatOpenAI

llm = ChatOpenAI(
    model="gpt-5.6"
)

agent = create_agent(
    model=llm,
    tools=[
```

```

        get_products,
        get_product,
        get_carts,
        get_users
    ],
    system_prompt="""
    You are an e-commerce assistant.

    Use the available tools whenever the user asks
    for information that must be obtained from the
    store API.
    """
)

```

Now the agent has four capabilities.



Part 8 — The LLM chooses the REST API

Now we can ask different questions.

Query 1

```

agent.invoke({
    "messages": [
        {
            "role": "user",
            "content": "What products are available?"
        }
    ]
})

```

The LLM can select:

```
get_products()
```

Query 2

```
Tell me about product 7.
```

The LLM can select:

```
get_product(product_id=7)
```

Query 3

```
Show me the shopping carts.
```

The LLM can select:

```
get_carts()
```

Query 4

```
Show me the users.
```

The LLM can select:

```
get_users()
```

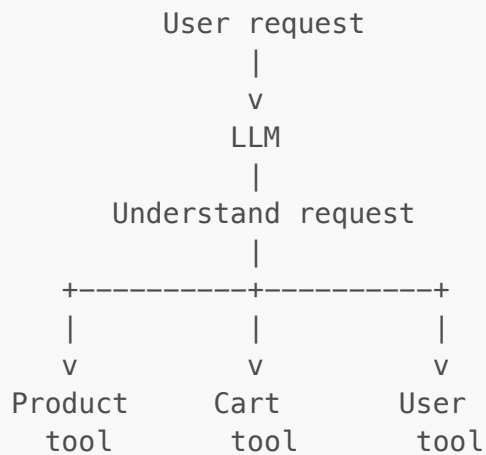
This is where the **agent** aspect becomes important.

We are no longer writing:

```
if "product" in query:  
    get_products()  
elif "cart" in query:  
    get_carts()
```

```
elif "user" in query:
    get_users()
```

Instead:



The LLM performs the routing.

Part 9 — Add a product search tool

Fake Store API supports retrieving products, but we can build a higher-level operation on top of it.

For example:

```
"Find products costing less than $50."
```

We can implement:

```
@tool
def find_products_by_max_price(max_price: float) -> str:
    """
    Find all products whose price is less than or equal
    to the specified maximum price.
    """

    response = requests.get(
        f"{BASE_URL}/products"
    )

    response.raise_for_status()

    products = response.json()
```

```
matching_products = [  
    product  
    for product in products  
    if product["price"] <= max_price  
]  
  
return str(matching_products)
```

Now this is interesting.

The REST API itself does not need to understand:

```
"Find products under $50"
```

Our tool does the transformation:

```
User  
|  
| Find products under $50  
v  
LLM  
|  
| find_products_by_max_price(50)  
v  
Python Tool  
|  
| GET /products  
v  
Fake Store API  
|  
| all products  
v  
Python  
|  
| filter price <= 50  
v  
LLM  
|  
v  
Answer
```

This demonstrates an important pattern:

A LangChain tool does not have to correspond one-to-one with a REST endpoint.

A tool can be a higher-level business operation that internally makes one or more REST API calls.

Part 10 — Complete example

Here is a complete example combining the concepts.

```
import requests

from langchain.agents import create_agent
from langchain_openai import ChatOpenAI
from langchain_core.tools import tool

BASE_URL = "https://fakestoreapi.com"

@tool
def get_products() -> str:
    """Get all products available in the store."""

    response = requests.get(
        f"{BASE_URL}/products"
    )

    response.raise_for_status()

    return response.text

@tool
def get_product(product_id: int) -> str:
    """Get details of a specific product using its product ID."""

    response = requests.get(
        f"{BASE_URL}/products/{product_id}"
    )

    response.raise_for_status()

    return response.text

@tool
def find_products_by_max_price(max_price: float) -> str:
    """Find products whose price is less than or equal to the specified maximum price."""

    response = requests.get(
        f"{BASE_URL}/products"
    )

    response.raise_for_status()
```

```
products = response.json()

matching_products = [
    product
    for product in products
    if product["price"] <= max_price
]

return str(matching_products)

@tool
def get_carts() -> str:
    """Get all shopping carts."""

    response = requests.get(
        f"{BASE_URL}/carts"
    )

    response.raise_for_status()

    return response.text

llm = ChatOpenAI(
    model="gpt-5.6",
    temperature=0
)

agent = create_agent(
    model=llm,
    tools=[
        get_products,
        get_product,
        find_products_by_max_price,
        get_carts
    ],
    system_prompt="""
You are an e-commerce assistant.

You have access to a store REST API through tools.

Use the tools when the user needs information
from the store.

Do not invent product information.
Base product-related answers on the API results.
"""
)

while True:
```

```
question = input("\nUser: ")

if question.lower() == "exit":
    break

result = agent.invoke({
    "messages": [
        {
            "role": "user",
            "content": question
        }
    ]
})

print(
    "\nAssistant:",
    result["messages"][-1].content
)
```

Part 11 — Try these requests

Once the program is running, try:

What products are available?

Then:

Give me details about product 3.

Then:

Find products costing less than 50 dollars.

Then:

Show me all shopping carts.

And finally something that requires no REST API:

What is an LLM?

The last request demonstrates an important agent behavior.

The agent does **not necessarily need to call a REST tool** when the requested information is already within the LLM's capabilities.

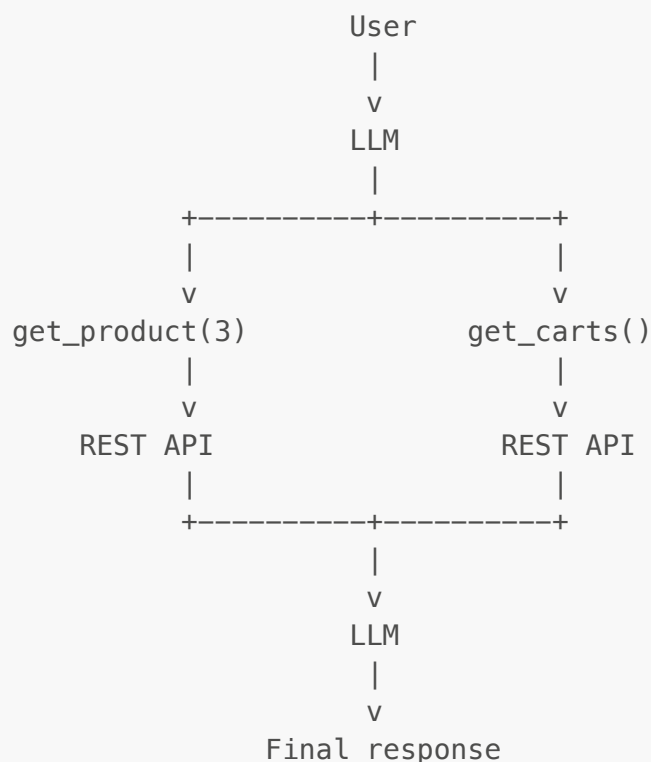
Part 12 — Multiple API calls in one request

The real power becomes apparent when a request requires multiple tools.

For example:

Give me information about product 3 and also show me the shopping carts.

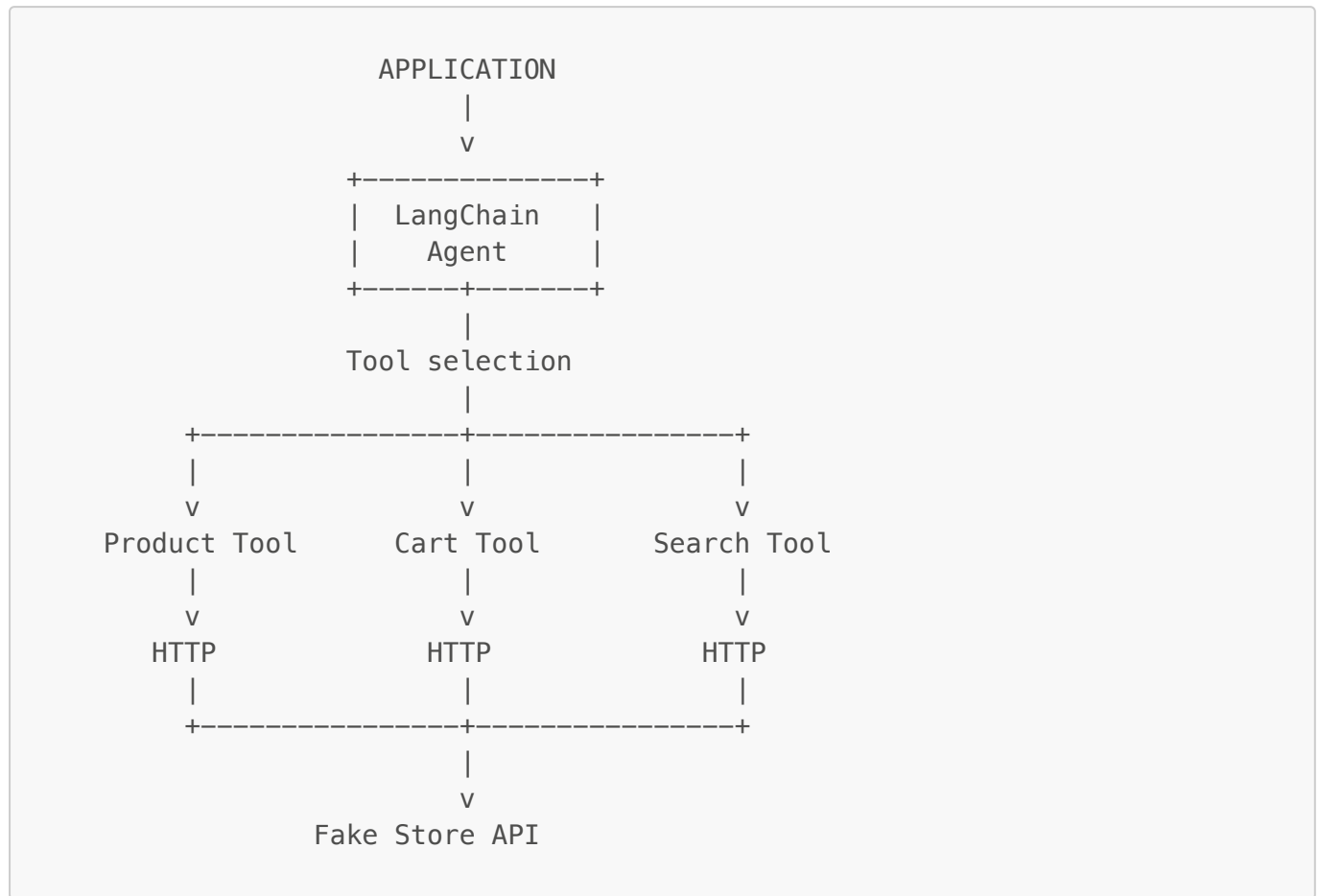
The agent may perform:



The current `create_agent()` implementation is designed around this model/tool loop: the model can return tool calls, the tools are executed, their results are added as tool messages, and the model is called again until there are no more tool calls. ([LangChain Reference](#))

Part 13 — REST API → Tool → Agent

The architecture can now be summarized as:



The important separation of responsibilities is:

Component	Responsibility
User	Expresses intent in natural language
LLM	Understands intent and decides whether a tool is needed
LangChain Agent	Manages the tool-calling loop
Tool	Defines an operation available to the LLM
Python function	Implements the operation
requests	Performs HTTP communication
Fake Store API	Provides the actual REST data

Part 14 — Why use tools instead of putting the API URL in the prompt?

Consider this approach:

You can access this API:

`https://fakestoreapi.com/products`

When the user asks about products,
figure out how to call it.

This is not a good architecture.

Instead:

```
@tool
def get_products():
    ...
```

The LLM receives a structured capability:

```
Tool name:
get_products

Description:
Get all products available in the store.

Arguments:
none
```

The model doesn't need to know how HTTP works.

It only needs to know:

```
"If I need product information, get_products is available."
```

This is one of the central ideas behind tool calling in LangChain: the model decides which tool to call, while the application owns the actual implementation and execution. ([LangChain](#))

Part 15 — REST API as an agent capability

It is useful to distinguish these three concepts:

REST API

```
GET /products/5
```

This is an external service.

LangChain Tool

```
@tool
def get_product(product_id: int):
    ...
```

This is the interface exposed to the LLM.

Agent

```
agent = create_agent(
    model=llm,
    tools=[get_product]
)
```

This allows the LLM to decide **when to use that capability**.

So:

```
REST API
  |
  | wrapped by
  v
LangChain Tool
  |
  | supplied to
  v
LLM Agent
  |
  | selected based on user intent
  v
Tool execution
```

That distinction becomes particularly important when moving from simple API integration to **agentic workflows**.

Suggested progression

A useful progression from here is:

1. REST API with requests
↓
2. REST call wrapped with @tool

-
- ```
graph TD; 3[3. One REST tool + LLM] --> 4[4. Multiple REST tools + LLM]; 4 --> 5[5. Agent chooses the appropriate API]; 5 --> 6[6. Agent performs multiple API calls]; 6 --> 7[7. Tools perform data transformation]; 7 --> 8[8. POST / PUT / DELETE tools]; 8 --> 9[9. Error handling]; 9 --> 10[10. Authentication]; 10 --> 11[11. REST API + RAG]; 11 --> 12[12. Multi-agent REST workflow];
```
3. One REST tool + LLM
  4. Multiple REST tools + LLM
  5. Agent chooses the appropriate API
  6. Agent performs multiple API calls
  7. Tools perform data transformation
  8. POST / PUT / DELETE tools
  9. Error handling
  10. Authentication
  11. REST API + RAG
  12. Multi-agent REST workflow

The Fake Store API is particularly suitable for this progression because it exposes products, carts, users, and authentication, along with POST/PUT/PATCH/DELETE operations. ([GitHub](#))

One important security distinction for the later exercises: **read-only GET tools are a good starting point**. Once you expose POST, PUT, PATCH, or DELETE operations, the tool can modify external state, so production agents should generally introduce validation and/or human approval before executing such operations.